C# REFRESHER

async, Task & the State Machine

Who creates the Task, what `async` really costs, and when dropping it goes wrong

Measured on .NET SDK 10.0.400, Windows 11 · Examples from the Driveway project · 11/09/2026

CONTENTS

1. The one-sentence version
2. Who creates the Task
3. Exceptions surface at different moments
4. What `async` generates
5. The measured cost
6. So where is the waste?
7. When dropping `async` goes wrong
8. Decision guide
9. In the Driveway codebase
10. Check yourself

1 The one-sentence version

> `async` is an instruction to the **compiler**, not part of the return type. It decides **who creates the Task**: the compiler, or you.

A caller can't tell the difference. `async Task M()` and `Task M()` look identical from the outside, which is why an interface can't declare `async` and why MassTransit's `IConsumer<T>.Consume` accepts either style.

2 Who creates the Task

With `async`: the compiler builds it

Your `return` is about the method's *result*. The compiler wraps it in the Task.

```
public async Task DeleteAsync()
{
    await DB.DeleteAsync<Item>(id);
    return;  // compiler completes it
}

public async Task<int> CountAsync()
{
    var n = await CountItemsAsync();
    return n; // an int, wrapped
}
```

Without `async`: you build it

An ordinary method whose return type happens to be `Task`. You return a real Task object.

```
public Task Delete()
{
    DoSyncWork();
    // already finished, cached
    return Task.CompletedTask;
}

public Task<int> Count()
{
    // already-finished Task<int>
    return Task.FromResult(5);
}
```

Mixing them up

CODE	RESULT	WHY
`async Task M() { return Task.CompletedTask; }`	CS1997 error	The compiler already makes the Task, so you can't return one as well.
`Task M() { return; }`	CS0126 error	The method promised a Task object and returned nothing.
`await` inside a non-`async` method	Compile error	Only `async` methods can await.
`async Task M() { }` with no `await`	CS1998 warning	Legal, but you asked for a state machine with nothing to wait for.

3 Exceptions surface at different moments

The one behavioural difference between the two styles:

```
public Task NotAsync()       { throw new Exception(); }
public async Task IsAsync() { throw new Exception(); }

var t1 = NotAsync();  // throws HERE, at the call
var t2 = IsAsync();   // no throw: you get back a Task already marked as failed
await t2;              // throws HERE, at the await
```

In an `async` method, exceptions are **stored inside the Task** and come out when you `await` it. In a plain `Task` method they throw immediately, before any Task exists. If you always `await` on the same line as the call, the two behave the same. It matters when you start a Task, do other work, and await it later.

4 What `async` generates

The compiler turns your method into a small structure that remembers where the method left off, plus a `MoveNext()` method that runs the next chunk of your code each time an `await` finishes. That's the **state machine**. Simplified:

```
struct <SeedData>d__0 : IAsyncStateMachine
{
    public int state;                       // which await we've reached
    public AsyncTaskMethodBuilder builder; // creates and completes the Task

    void MoveNext()
    {
        try   { /* your method body, split at each await */ }
        catch (Exception e) { builder.SetException(e); return; }
        builder.SetResult();
    }
}
```

KEY IDEA

In **Release** builds the state machine is a `struct` on the stack. It only moves to the heap when an `await` really has to pause and resume later. In **Debug** builds it's a `class`, so the debugger can inspect it, and a class is always allocated.

5 The measured cost

Average heap bytes allocated per call, over 200,000 calls each:

METHOD	RELEASE	DEBUG
`async Task`, no await	**0 B**	32 B
`Task` → `Task.CompletedTask`	**0 B**	**0 B**
`async Task<int>` returning `1`	**0 B**	32 B
`async Task<int>` returning `42`	72 B	104 B
`Task.FromResult(42)`	72 B	72 B
`async Task` with a **real** await (`Task.Yield()`)	96 B	120 B

Measured with `GC.GetTotalAllocatedBytes(precise: true)`. Exact numbers can shift between .NET versions; the pattern doesn't. Source code in the appendix.

- **Rows 1 and 2 are both zero in Release.** When an `async Task` finishes without ever pausing, .NET hands back the same cached completed Task that `Task.CompletedTask` gives you.
- **Row 3 is zero, row 4 isn't.** .NET keeps ready-made completed `Task<T>` objects for a few common results (`true` / `false`, small integers such as `1`, `null`). `42` isn't one of them.
- **Row 5 equals row 4.** `Task.FromResult(42)` allocates the same 72 bytes, so dropping `async` from a method returning `42` saves nothing.
- **Row 6 is `async` doing its job:** saving its place so it can resume later. That cost is the point, not waste.

6 So where is the waste?

In Release, an `async` method with no awaits costs **CPU time, not memory**: the extra generated code, the `try/catch`, and calls into the builder. That's a few nanoseconds per call. It matters in a hot loop or library code called millions of times, and disappears in a controller or consumer that also talks to a database.

The better reasons to drop `async` when there's nothing to await:

- **Readability.** Warning CS1998 exists mainly so a *missing* `await` stands out. An `async` method with no `await` is often a bug where you forgot one, and removing `async` makes it clear the method is synchronous on purpose.
- **Debug builds** really do allocate on every call.

7 When dropping `async` goes wrong

Dropping `async` is safe when the body is synchronous. It's risky when you're passing along a Task you didn't create yourself, because `using` and `try` end the moment the Task is **returned**, not when it **finishes**.

PITFALL 1 · DISPOSED TOO EARLY

```
public Task<string> GetPageAsync()
{
    using var client = new HttpClient();
    return client.GetStringAsync(url); // client disposed before the request finishes
}
```

PITFALL 2 · A CATCH THAT NEVER CATCHES

```
public Task DeleteItemAsync()
{
    try   { return DB.DeleteAsync<Item>(id); }
    catch (Exception ex) { _logger.LogError(ex, "..."); throw; } // misses async failures
}
```

The fix for both is the same: add `async` back and `await` the call. Then the `using` and the `try` stay in effect until the work actually finishes.

8 Decision guide

SITUATION	DO THIS
Body does synchronous work, no awaits	Drop `async`; return `Task.CompletedTask` or `Task.FromResult(...)`
Body awaits anything	Keep `async`
Body is only `return OtherAsync();` with nothing around it	Either works; dropping `async` is slightly cheaper
Pass-through inside a `using`, `try` or `lock`	Keep `async` and `await` it
Hot path that often returns an already-known value	Consider `ValueTask<T>`: no allocation when the result is ready

9 In the Driveway codebase

ALREADY APPLIED
`AuctionService/Data/DbInitializer.cs`

`SeedData` calls only synchronous methods (`Migrate()` , `SaveChanges()`), so it was changed from `async Task` to `Task` in commit `eab058c` . Both exits return `Task.CompletedTask` :

```
private static Task SeedData(AuctionDbContext context)
{
    context.Database.Migrate();
    if (context.Auctions.Any())
        return Task.CompletedTask; // already seeded
    // ... build and add the seed auctions ...
    context.SaveChanges();
    return Task.CompletedTask;
}
```

ALREADY APPLIED
`AuctionService/Consumers/AuctionUpdatedFaultConsumer.cs`

`Consume` only logs, so it's a plain `Task` method ending in `return Task.CompletedTask;` rather than an `async` method with nothing to await, which gave warning CS1998.

Key takeaways

- `async` decides who creates the Task. Callers can't see the difference.
- Exceptions from `async` methods surface at the `await` ; from plain `Task` methods, at the call.
- In Release, `async` with no awaits allocates nothing. The real costs show up when an await actually pauses, and in Debug builds.
- Drop `async` for synchronous bodies. Keep it whenever a `using` , `try` or `lock` surrounds the Task you return.

10 Check yourself

1. Which of these compile?
   (a) `async Task M() { return Task.CompletedTask; }`
   (b) `Task M() { return; }`
   (c) `Task M() => Task.CompletedTask;`
2. In a Release build, how many heap bytes does an `async Task` method with no `await` allocate per call?
3. Given `Task NotAsync() { throw new Exception(); }`, which line of `var t = NotAsync(); await t;` throws?
4. Why does `GetPageAsync()` in Pitfall 1 fail, and what's the fix?
5. Would removing `async` from `async Task<int> Get() => 42;` save an allocation? If not, what would?

Answers are on the last page.

Appendix · how the numbers were measured

Run with `dotnet run -c Release` and `dotnet run -c Debug` in a fresh console project.

```
const int N = 200_000;

Report("async Task, no await",         () => AsyncNoAwait());
Report("Task => Task.CompletedTask",   () => PlainCompleted());
Report("async Task<int> returning 1",  () => AsyncReturn1());
Report("async Task<int> returning 42", () => AsyncReturn42());
Report("Task.FromResult(42)",          () => FromResult42());
Report("async Task with a REAL await", () => AsyncYield());

static void Report(string label, Func<Task> f)
{
    for (int i = 0; i < 2_000; i++) f().GetAwaiter().GetResult(); // warm up
    long before = GC.GetTotalAllocatedBytes(precise: true);
    for (int i = 0; i < N; i++) f().GetAwaiter().GetResult();
    long perCall = (GC.GetTotalAllocatedBytes(precise: true) - before) / N;
    Console.WriteLine($"{label,-32} {perCall,4} bytes/call");
}

static async Task      AsyncNoAwait()   { }
static Task            PlainCompleted() => Task.CompletedTask;
static async Task<int> AsyncReturn1()   => 1;
static async Task<int> AsyncReturn42()  => 42;
static Task<int>       FromResult42()   => Task.FromResult(42);
static async Task      AsyncYield()     { await Task.Yield(); }
```

Answers to “Check yourself”

1. Only (c). (a) is CS1997, (b) is CS0126.
2. Zero. .NET returns a cached completed Task, and the state machine stays on the stack.
3. `var t = NotAsync();` — a plain `Task` method throws before any Task exists.
4. The `using` disposes the `HttpClient` as soon as the Task is returned, while the request is still running. Make the method `async` and `await` the call.
5. No: `Task.FromResult(42)` allocates the same 72 bytes. Returning `ValueTask<int>` would avoid the allocation.
